

Model Binding Using FromHeader in ASP.NET Core Web API

Back to: [ASP.NET Core Web API Tutorials](#)

Model Binding Using FromHeader in ASP.NET Core Web API

In this article, I will discuss How to Implement **Model Binding Using FromHeader in ASP.NET Core Web API** Application with Examples. Please read our previous article discussing [Model Binding Using FromRoute in ASP.NET Core Web API](#) with Examples.

FromHeader Attribute in ASP.NET Core Web API

The FromHeader attribute in ASP.NET Core Web API binds a parameter in an action method to the value of a specific HTTP request header. It tells the framework that the value of the method parameter should be obtained from the specified HTTP header in the incoming request. This is useful for reading metadata, version information, authentication tokens, or other information transmitted in HTTP headers. If you go to the definition ofFromBodyAttribute, you will see the following signature.

```
public class FromHeaderAttribute : Attribute, IBindingSourceMetadata, IModelNameProvider
{
    public BindingSource BindingSource => BindingSource.Header;

    public string? Name { get; set; }
}
```

The FromHeaderAttribute specifies that a parameter or property should be bound using the request headers.

How Does the FromHeader Attribute Work in ASP.NET Core Web API?

When a request is made to an API endpoint, ASP.NET Core's model binding system inspects the request and maps the data to action parameters. If a parameter is decorated with the FromHeader attribute, the model binder looks for a matching header in the HTTP request. If it finds a header with the same name as the parameter (or the name specified in the attribute's Name property), it binds the value of that header to the parameter.

```
[HttpGet]
public IActionResult GetUser([FromHeader] string authToken)
{
    // The value of the authToken parameter will be set from the "authToken"
    header in the request
    return Ok(authToken);
}
```

```
}
```

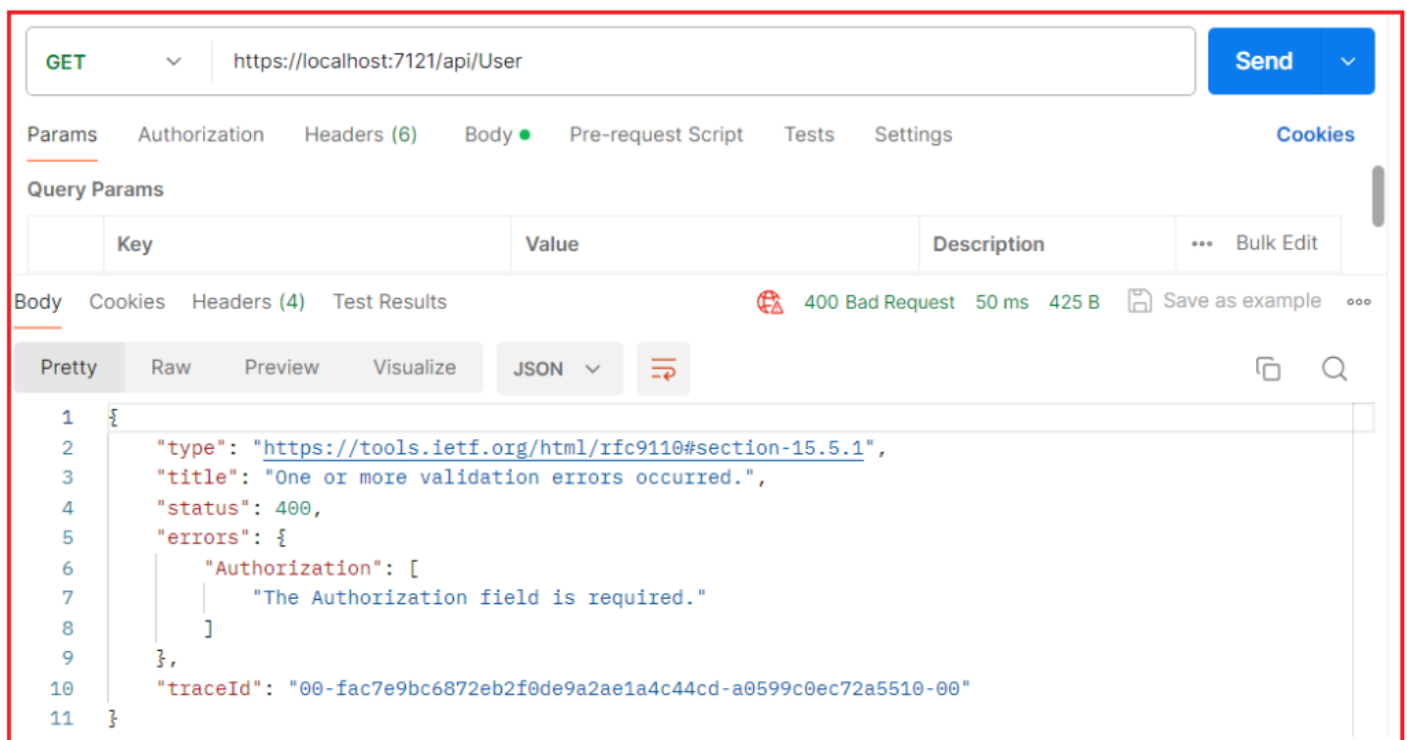
Example to Understand FromHeader Attribute in ASP.NET Core Web API:

When you decorate a parameter with the [FromHeader] attribute in an action method, ASP.NET Core will attempt to find a header of the same name as the parameter in the incoming HTTP request and bind its value to the parameter. For a better understanding, please modify the User Controller as follows. We have decorated the Authorization parameter with the FromHeader Attribute. The ASP.NET Core Framework will map this parameter with the incoming HTTP Request's Authorization header value.

```
using Microsoft.AspNetCore.Mvc;
namespace ModelBinding.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class UserController : ControllerBase
    {
        [HttpGet]
        public IActionResult GetResource([FromHeader] string Authorization)
        {
            // Implementation
            if(Authorization == null)
            {
                return BadRequest("Authorization Token is Missing");
            }

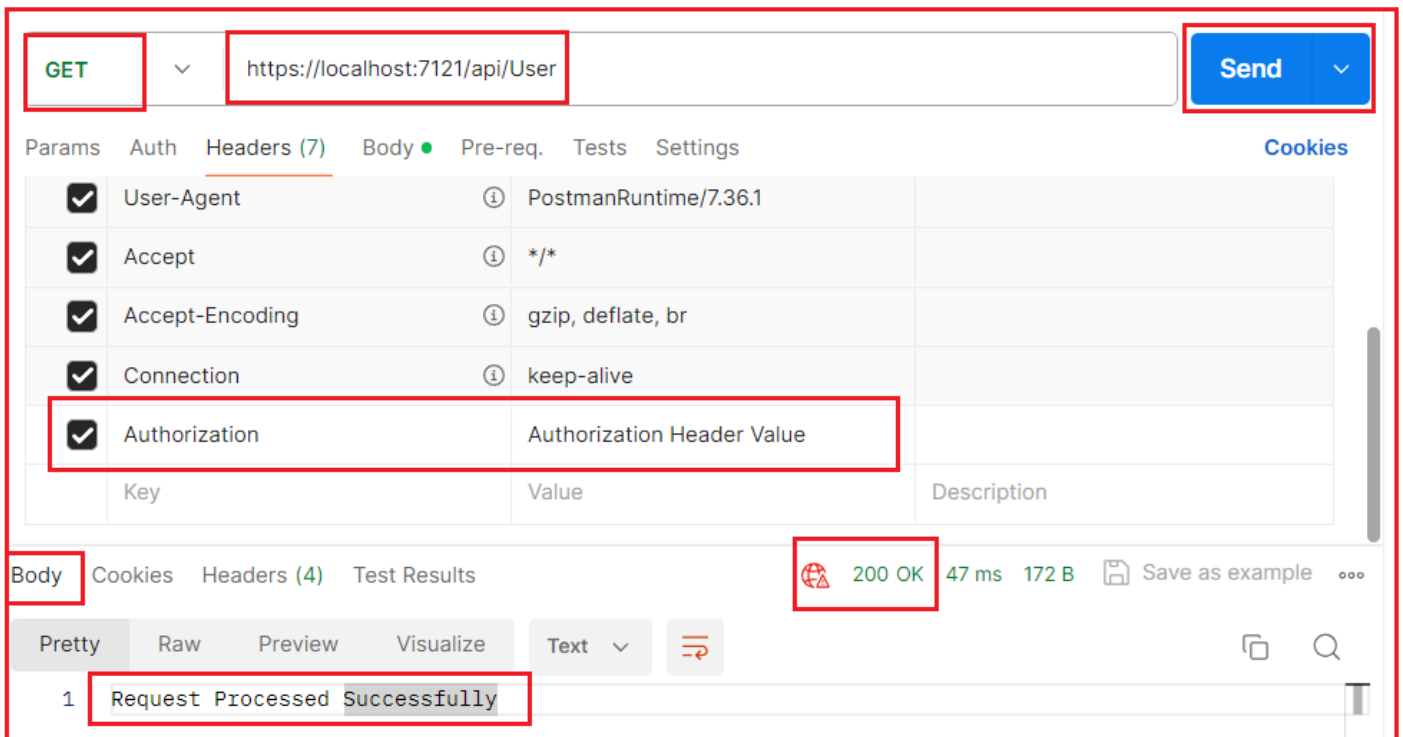
            return Ok("Request Processed Successfully");
        }
    }
}
```

If we send the authorization header value while making the HTTP request, we will get a 200 OK status code; otherwise, if the authorization header is missing in the HTTP request, it will return a 400 Bad Request. Now, run the application and send an HTTP GET request to the above endpoint (**/api/user**) without an Authorization header, and you should get the following response. Please change the port number where your application is running:



Suppose you are wondering why the custom error message we passed to the `BadRequest` method is not returning in the response body. This is because of the `ApiController` Attribute, which is decorated at the Controller level. This `ApiController` Attribute will check the model state. Suppose it is not valid (checking the values of the mandatory parameter, the parameter which is non-nullable). In that case, it will simply return 400 Bad Requests without executing the Controller action method. The above response is the default Bad Request response set by the `ApiController` Attribute.

Now, if you set the `Authorization` header attribute in the request header, then you will get the 200 OK Status code as shown in the below image:



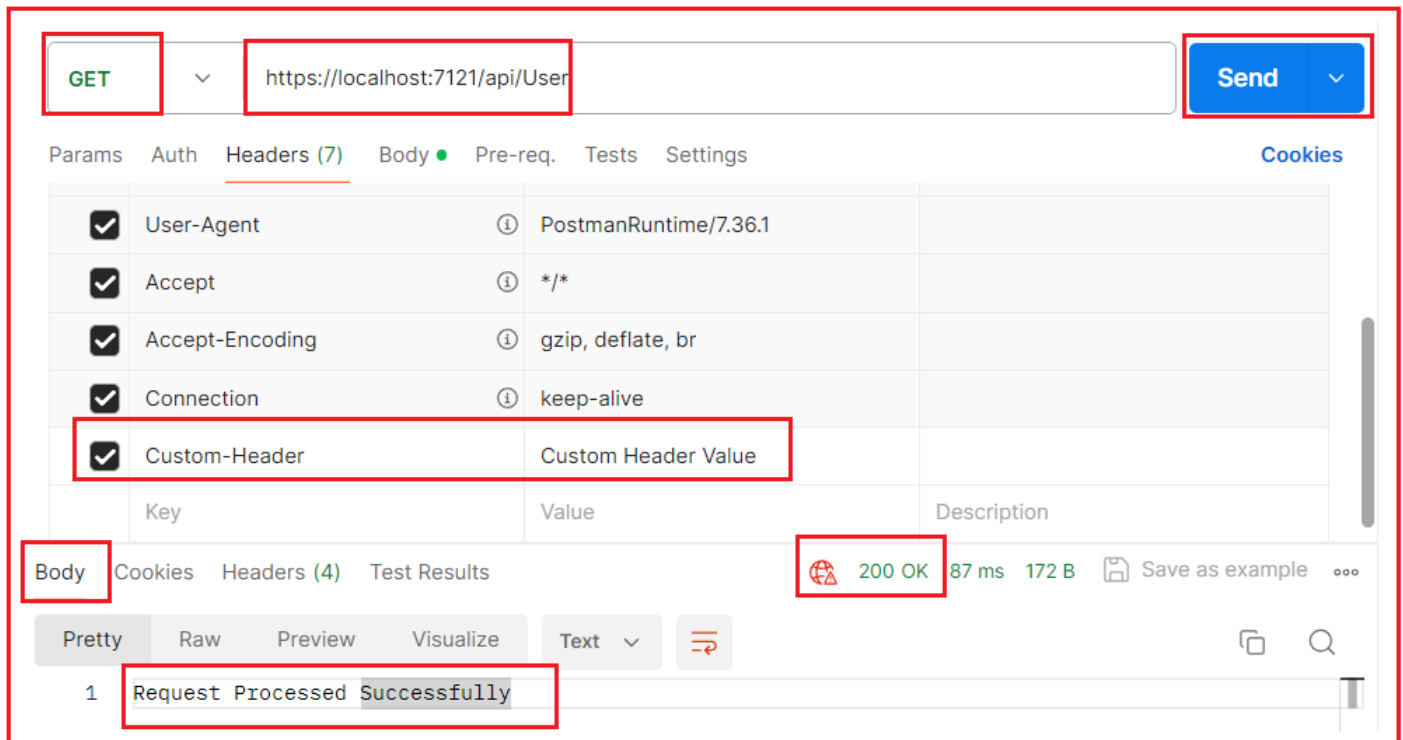
Giving Different Names to Method Parameters:

The Name property of the FromHeader attribute is used when the name of the header in the HTTP request does not match the parameter name in the action method. You can specify the exact header name using the Name property. To better understand, please modify the User Controller as follows.

```
using Microsoft.AspNetCore.Mvc;
namespace ModelBinding.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class UserController : ControllerBase
    {
        [HttpGet]
        public IActionResult GetResource([FromHeader(Name = "Custom-Header")]
string customHeader)
        {
            if(customHeader == null)
            {
                return BadRequest("Custom-Header is Missing");
            }

            return Ok("Request Processed Successfully");
        }
    }
}
```

In the above example, the **GetResource** method expects a custom header named Custom-Header in the incoming HTTP request. The FromHeader attribute specifies that the customHeader parameter should be populated with the value of this header. Now, run the application and access the above endpoint (**api/user**) with a GET HTTP Request, and it should work as expected, as shown in the below image:



When Should We Use the FromHeader Attribute in ASP.NET Core Web API?

We need to use the FromHeader attribute when extracting values from HTTP request headers and using them in action methods. This is commonly used for:

- **Authentication Tokens:** Clients typically pass tokens in the request headers when using token-based authentication, such as JWT (JSON Web Tokens). You can use FromHeader to extract these tokens directly in your action method parameters.
- **Versioning Information:** API versioning can be implemented through headers to specify which version of the API the client intends to use. Using FromHeader, you can capture the version information directly and route the request to the appropriate logic based on the requested API version.
- **Custom Metadata:** Sometimes, applications require additional metadata for processing requests that don't fit in the body or URL. Examples include request IDs for tracing, client-specific flags, or feature toggles. FromHeader can be used to capture such metadata efficiently.

In the next article, I will discuss [Model Binding Using FromBody in ASP.NET Core Web API](#) with Examples. In this article, I will try to explain **model binding using FromHeader in ASP.NET Core Web API** with examples. I hope you enjoy this article, "Model Binding Using FromHeader in ASP.NET Core Web API."



Dot Net Tutorials

About the Author: *Pranaya Rout*

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information